Assignment 2

Roll No. 230217

Setup

Everything is built with `make` and was tested on GPU3 (NVIDIA A40, CUDA 12.6).

1 Problem 1: 3D convolution

```
make problem1
./problem1/problem1.out   # N: edit Input_Width (1<<6, 1<<7, 1<<8) in 230217-prob1.cu
```

- **Basic kernel:** one thread per output, $8 \times 8 \times 8$ blocks, input and filter read from global memory.
- **Optimised kernel:** (a) a $12 \times 12 \times 12$ shared-memory tile per block (the 8^3 outputs plus a halo of 2, ghost cells written as 0); (b) the filter in `__constant__` memory; (c) the innermost loop unrolled by hand.

`check_result` reports "No differences found" for both kernels at $N = 64, 128, 256$.

N	CPU	Basic kernel	Opt kernel	Opt vs basic	Basic e2e	Opt e2e
64	38	0.047	0.026	1.80×	0.80	0.76
128	337	0.293	0.143	2.04×	4.19	4.00
256	2754	2.321	1.106	2.10×	26.94	24.48

Table 1: Problem 1, ms, best of 6 runs (each averages 10 launches); e2e = with copies.

The optimised kernel is 1.80–2.10× faster than the basic one, because the tile reads each input value from global memory once per block instead of up to 125 times. End to end the two are almost equal, since the copies dominate.

2 Problem 2: grid search

```
make problem2                # builds problem2-v1.out, -v2.out, -v3.out from 230217-prob2.cu
cd problem2; ./problem2-v1.out   # also v2, v3; reads disp.txt, grid.txt; writes results-vN.txt
```

The grid has $13^{10} \approx 1.4 \times 10^{11}$ points. A point is one 64-bit number, so the matches can be sorted and written in the loop order of `problem2-v0.c`.

- **(i) Simple port:** the host loops over variables 1–2 (169 launches); one thread per combination of variables 3–6 loops over variables 7–10 and recomputes all ten rows for every point.

- **(ii) Optimised:** the same exhaustive search with running row sums reused across the inner variables, an early exit at the first failing row, and only variable 1 on the host (13 launches of $13^5 = 371{,}293$ threads). Shared-memory tiling and prefetching are not used: the 150 input numbers are in constant memory.
- **(iii) Thrust:** slices of 2^{28} points; `transform` writes a pass/fail flag per point, `count_if` sizes the output and `copy_if` collects the matches.

All three match `problem2-v0.c` byte for byte on 6^{10} points (2071 results), 8^{10} (9316) and the full grid (11,608).

Version	Best (s)	Worst (s)	vs (i)	vs CPU
CPU `problem2-v0.c`	676.2	—	—	1×
(i) simple port	36.13	80.9	1.00×	18.7×
(ii) optimised	6.28	14.7	5.76×	107.7×
(iii) Thrust	34.89	91.2	1.04×	19.4×

Table 2: Problem 2, full grid, `Total time` printed by each program; 10 interleaved runs per version, the best started with an idle GPU, the worst with other jobs running.

(ii) is 5.76× faster than (i) (reused sums, early exit, larger launches). (i) and (iii) are about equal on an idle GPU; I did not profile why. Under load all times grew by 2.2–2.6× and (i) and (iii) changed places.

3 Problem 3: counting Bloom filter

```
make problem3                # builds problem3-vanilla.out and problem3-blocked.out
cd problem3; ./problem3-vanilla.out N $PWD/insert_100000.bin \
    $PWD/search_100000.bin $PWD/delete_100000.bin      # same for problem3-blocked.out
```

The filter is an array of N 8-bit counters and every key maps to $k = 3$ counters:

```
insert(key):   for i in 1..3: counter[hash_i(key)] += 1                  # atomicAdd
search(key):   return all(counter[hash_i(key)] > 0 for i in 1..3)
delete(keys):  kernel 1: found[j] = search(keys[j])         # before any decrement
               kernel 2: if found[j]:                      # atomicCAS, never below 0
                             counter[hash_i(keys[j])] -= 1   for i in 1..3
blocked:       hash_i(key)  ->  hash_4(key, N/128)*128 + hash_i(key, 128)       # i = 1..3
```

Four counters share one 32-bit word (CUDA has no 8-bit atomics); counter p is byte $p\%4$ of word $p/4$. One thread handles one key, and duplicates in a trace are removed on the host first. `hash1`–`hash4` are separate functions returning a value in $[0, n)$. The blocked variant confines a key to one 128-counter block (one cache line). `check_result()` replays the operations on a host copy of the counters and a `std::set` of members, compares every counter after insert and after delete, and counts false negatives and false positives (false positives over all searches). Traces come from `trace_generator.cpp` (n = 100,000 in the submission); with 9.9 million keys per operation all checks also pass.

In every run of Table 3, 0 counters differ from the host model and there are 0 false negatives.

		vanilla		blocked	
Trace	N	insert	delete	insert	delete
$n = 10^5$	10^6	0.547%	0.389%	0.662%	0.554%
$n = 2 \cdot 10^6$	$2 \cdot 10^7$	0.583%	0.390%	0.665%	0.500%
$n = 10^5$	$3 \cdot 10^5$	8.42%	7.73%	8.52%	7.91%

Table 3: false-positive rate after the insert and after the delete (a third of the searches are for never-inserted keys, two thirds after the delete).

The blocked variant has more false positives in every row, because keys in the same block compete for its 128 counters.

Deleting never-inserted keys. With 10,020 never-inserted keys among the 43,354 deleted keys ($N = 3 \cdot 10^5$) the GPU counters still equal the host model in all 3 runs. 2469 (vanilla) and 2549 (blocked) of those keys looked present and were decremented, causing 2025 and 1989 false negatives for real members; no counting Bloom filter can avoid this.

Profiling (Nsight Compute, first launch of each kernel, $n = 2 \cdot 10^6$, $N = 2 \cdot 10^7$):

	Vanilla		Blocked	
Metric	insert	search	insert	search
Kernel duration	1.46 ms	625 μs	777 μs	371 μs
DRAM traffic	689 MB	347 MB	431 MB	220 MB
Memory bandwidth utilisation (DRAM, % of peak)	67.9	79.8	79.8	85.5
Arithmetic intensity (FLOP/byte)	0	0	0	0
integer instructions per DRAM byte	0.21	0.44	0.34	0.71
Achieved occupancy (%)	69.8	81.2	72.2	81.8
Warp efficiency (active threads / 32)	32/32	32/32	32/32	32/32
threads not predicated off (of 32)	29.9	30.1	30.7	30.8
L2 cache hit rate (%)	9.7	10.4	36.5	19.9
Long-scoreboard stalls (% of warp-active time)	22.6	50.9	22.2	57.5

Table 4: Nsight Compute metrics.

There are no floating-point instructions, so the arithmetic intensity is 0 FLOP/byte; 32 threads are active per warp (about 30 not predicated off). Top three bottlenecks reported by the profiler:

1. **Uncoalesced global accesses:** 84% (insert) and 83% (search) of the L2 sectors are excess; search uses only 5.1 of every 32 bytes fetched.
2. **Memory-bound, long-scoreboard stalls:** DRAM at 68–85% of peak; a search warp waits 288 (vanilla) / 202 (blocked) cycles for loads.
3. **Low issue rate and occupancy:** an instruction every 35–146 cycles, occupancy 70–84%.

Blocking makes insert 1.9× and search 1.7× faster with 36–37% less DRAM traffic (the three counters of a key share a cache line), at the price of more false positives.